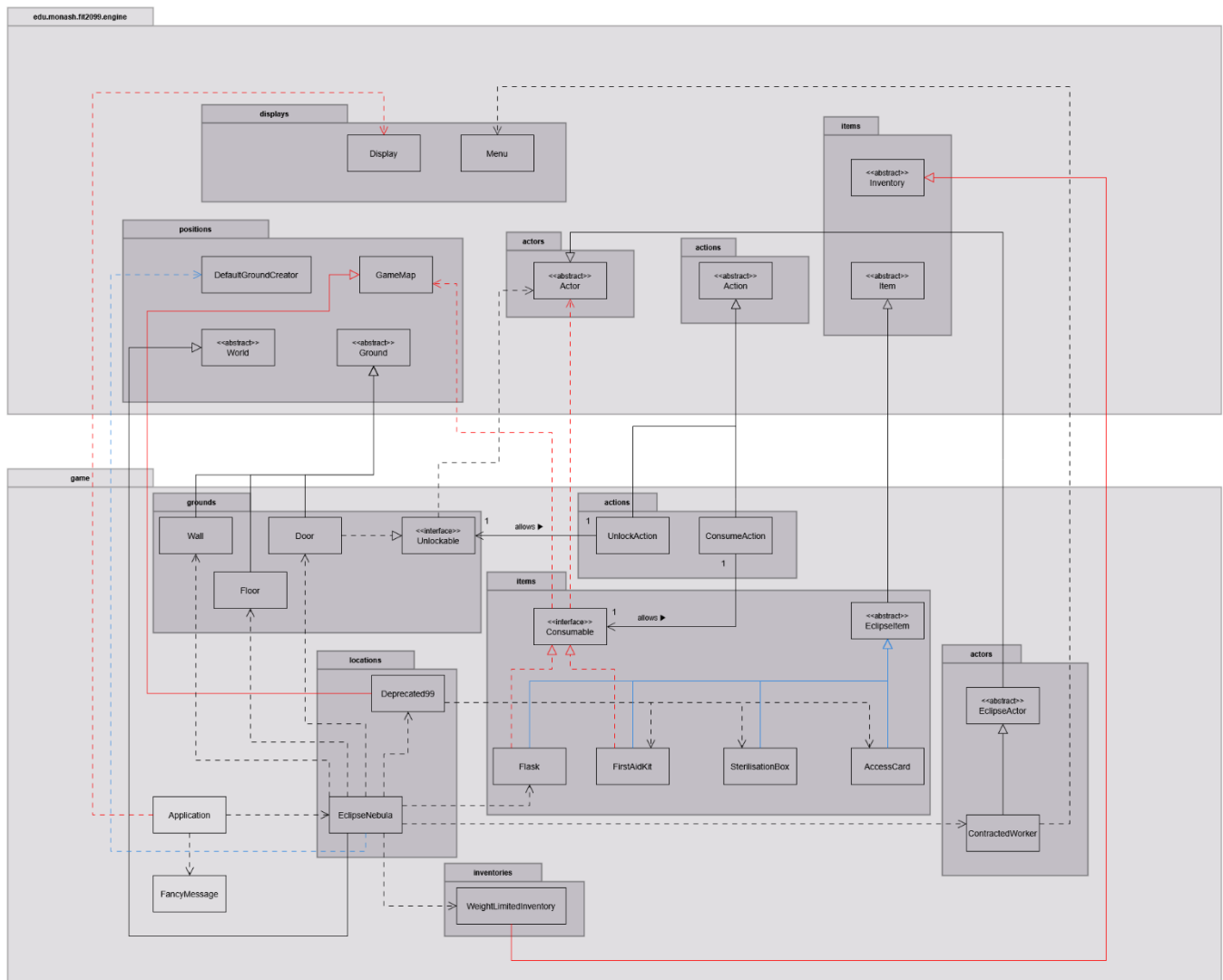
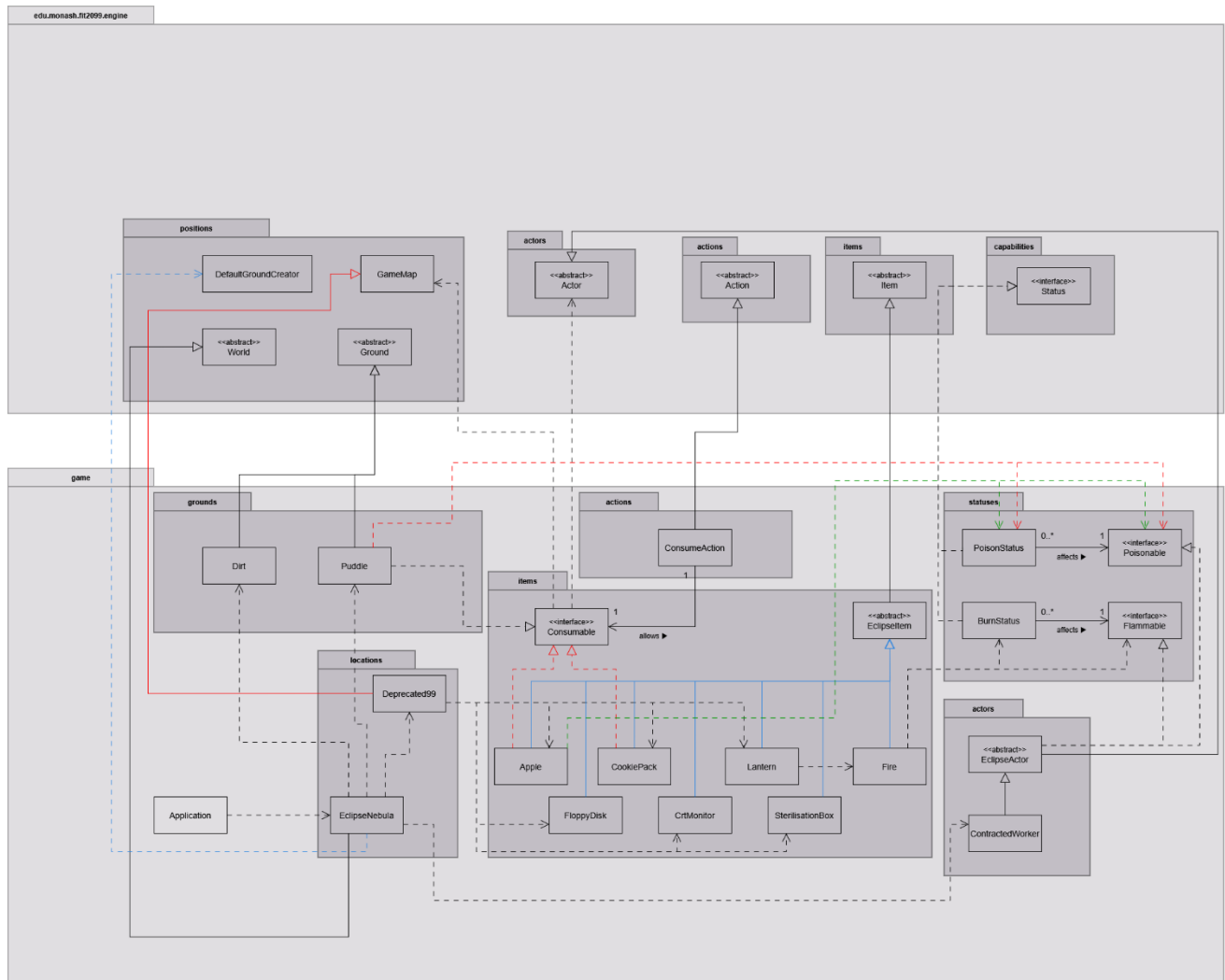
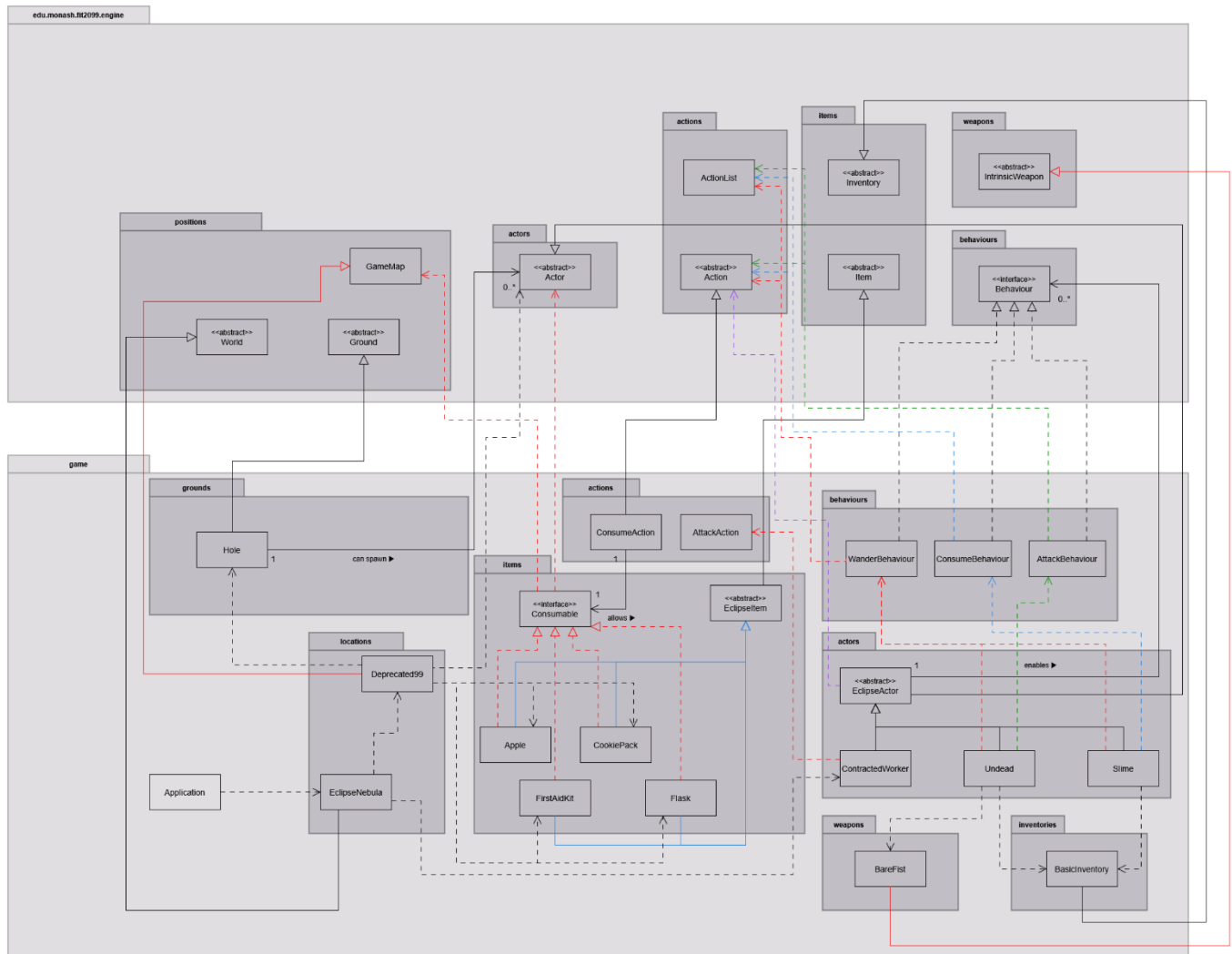




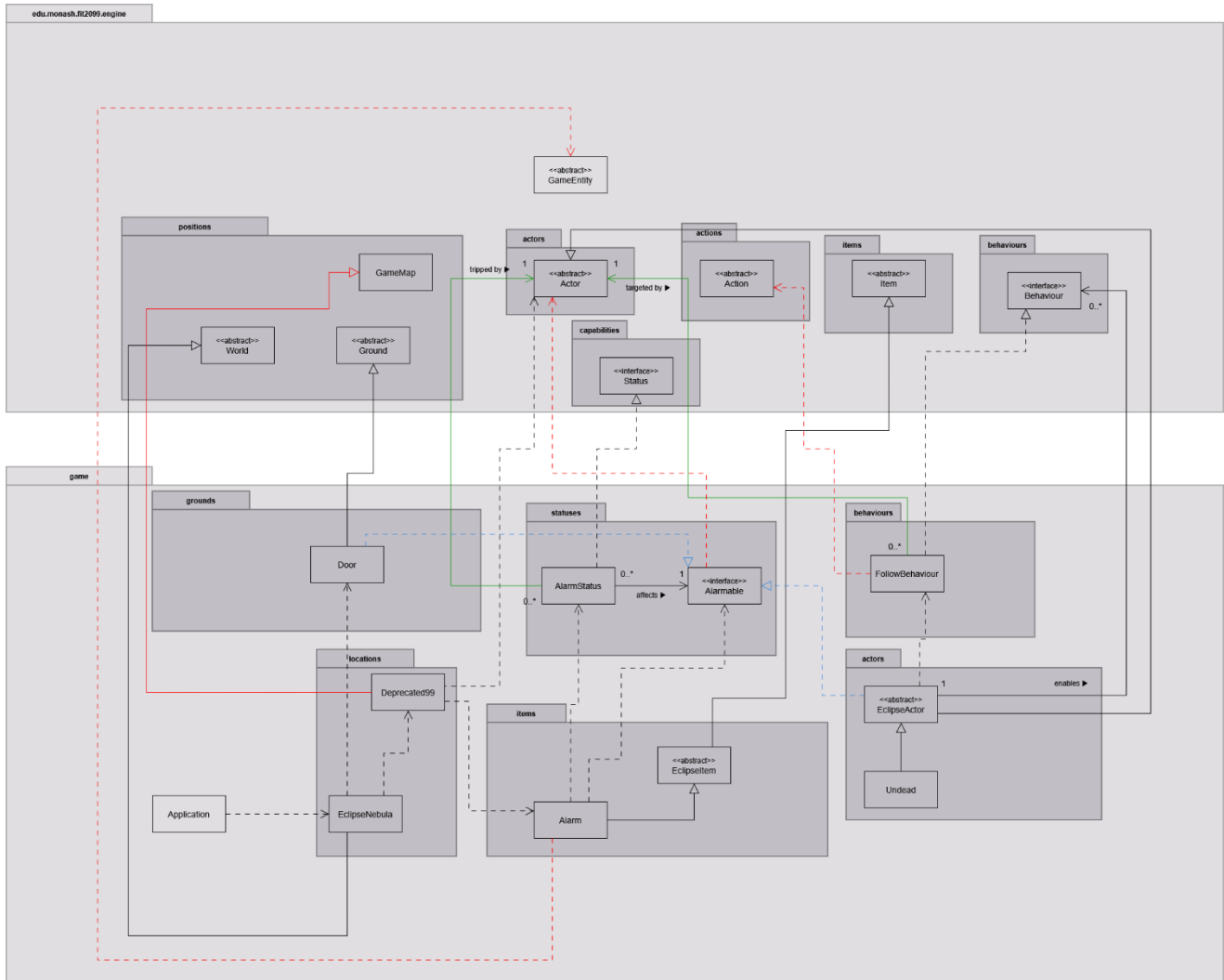
Requirement 2



Requirement 3



Requirement 4



Each UML diagram is available in .png format on their own for better clarity.

The design rationale will aim to cover the major decisions I've made across the 4 requirements.

Requirement 1

Decision 1: Making each individual item in the game have a weight value.

Chosen design: Create a new abstract class for the game, `EclipseItem`, with weight being one of the attributes. All items in the game will then inherit from `EclipseItem`, and specify their weight when calling their super constructor.

Advantages	Disadvantages
Allows to standardize common attributes that items in this game will have.	This adds a new abstraction layer. Having too many layers of abstraction can overcomplicate code, leading it to be less maintainable.

Alternative design: Don't create a new abstract class, and instead have each item's own constructor initialize their weight as an attribute.

Advantages	Disadvantages
Keeps the abstraction in the game code to a minimum. All items already inherit from the engine's <code>Item</code> class.	Adding new items to the game will require duplicating code in the constructor to initialize attributes like weight.
-	When adding a new item to the game, a developer may forget to add common attributes to the constructor, leading to unintended behaviour.

The chosen design was used as it allows standardization of common attributes* of the items in the game, with weight being the example, since all items in the requirements had weight. Furthermore, the engine code does not

make items portable by default. Each item in the requirements is supposed to be portable. This can be solved by making it portable in EclipseItem's constructor (items that aren't portable can make themselves not portable in their own constructor). This design avoids unnecessary code duplication, adhering to the DRY principle. Although this does come at a cost of an increased layer of abstraction, which goes against the KISS principle, standardizing common attributes or behaviours outweighs that drawback as adding new classes via the alternative design would be more error-prone (suppose someone forgot to give a new item a weight, or made it portable).

*Later on, weight actually uses an enum and entity's statistics, as provided by the engine. This is to show integration with the engine.

Decision 2: Making Flask and FirstAidKit able to be used/consumed by the worker.

Chosen design: As they are similar, create an interface, Consumable, and let both of those items implement it. Then, make ConsumeAction consume said consumable, with Flask and FirstAidKit implementing its own logic to being consumed.

Advantages	Disadvantages
New classes that can be consumed can just implement Consumable and define their own logic for being consumed.	Consumables would have an extra responsibility of handling the logic of being consumed (consumedBy method), as well as when it can be consumed (allowableActions method).

Alternative design: Have separate ConsumeFlaskAction and ConsumeKitAction to allow consuming each item respectively. The actions themselves will handle the logic.

Advantages	Disadvantages
The action becomes fully responsible for what happens, rather than the item itself having to take on extra responsibilities.	Having an action associated with one specific item, which performs similarly, introduces unnecessary code duplication especially when new items are added with can be consumed.

The chosen design was used as it is more easily extensible for new items that will have similar logic to being consumed. By using the Consumable interface, ConsumeAction can be associated with a Consumable instead of each individual class (Flask, FirstAidKit etc.). ConsumeAction can do what it wants with the Consumable item, no matter if it's a Flask or FirstAidKit, demonstrating the Liskov Substitution Principle (LSP) as those classes fully substitute Consumable. Unlike the alternative, this allows polymorphism, increases code reusability and reduces unnecessary code duplication, adhering to DRY as well.

It could also be argued that each consumable item itself should be the one defining what happens when it's consumed, rather than its action. So it's not a violation of the Single Responsibility Principle (SRP) with that consideration.

Decision 3: Implementing the ContractedWorker's playTurn method.

The partial implementation given was essentially ignoring the actions the engine provides for the player, instead creating its own actions, so that was changed to use what's given by the engine, showing integration with the existing system. Decision 2 being made was beneficial to this too as Consumable items gave their allowed actions when available for the player to

choose. This approach uses what's already given instead of unnecessary code duplication.

Requirement 2

Decision 1: Adding the moon 99-Deprecated to the game, which is a GameMap.

Chosen design: Have 99-Deprecated as a separate class*, let it inherit GameMap, and define its own map structure, and items scattered around.

*called Deprecated99 since Java doesn't allow the first character to be a number for class names

Advantages	Disadvantages
Each moon (GameMap) in this game will have its own class and structure. A particular moon can be easily accessed to make changes to (e.g. adding more sample items) if needed.	Requires adding more classes into the system. This could make it look complex.

Alternative design (provided by partial implementation): Have EclipseNebula (subclass of World) define 99-Deprecated's map structure and loose items.

Advantages	Disadvantages
Seems easier to keep all the data of the GameMaps contained within one class.	EclipseNebula is defining the structure and items for each of its maps, which is a sign of taking on too many responsibilities.
-	May be hard to change the structure of an existing gameMap, as you need to find it in EclipseNebula's very long initialise method.

The chosen design was used as it allows better organization of the GameMaps in the game. The GameMaps themselves are responsible for defining what their ground tiles and items look like, while EclipseNebula only needs to gather the GameMaps together so the game can be played. This adheres to the SRP as both EclipseNebula and the GameMaps have their own distinct responsibility. As a result, the classes are more cohesive and the code is more maintainable as it is easier to locate in which class changes should be made if needed.

Decision 2a: Adding burn and poison status effects for all actors in the game.

Chosen design (like the mars example): Create interfaces Poisonable and Flammable, let EclipseActor implement them. When PoisonStatus or BurnStatus is called each tick (turn), let the classes implementing Poisonable/Flammable define what happens to themselves when poisoned or burned each turn.

Advantages	Disadvantages
Other entities (e.g. items) can easily implement the interfaces and define their own way of handling the status effects.	Adding interfaces makes the code more complex.

-	The current implementation of applying status effects to actors uses downcasting in the engine's code, which is a code smell.
---	---

Alternative design: Don't create Poisonable or Flammable interfaces. Instead, BurnStatus/PoisonStatus will directly hurt (decrease health) the affected actor each turn. Due to how the engine works, entities (actors/grounds/items) are the ones that have status effects. Entities lack a hurt method, so instanceOf and downcasting to actor could be used.

Advantages	Disadvantages
Simpler to implement as all actors currently can be burned and poisoned.	Cannot easily be extended to apply status effects to non-actors or exclude actors with immunities (e.g. fireproof). This would require lots of if/elseif checks and/or downcasting to specific concrete classes.
-	Downcasting is even more of a code smell when the engine isn't used.

The chosen design was used due to it being more easily extensible. New classes implementing it only need to define how to handle the status effect, and it can be applied to them. As adding this functionality is achievable without needing to modify any other classes, this adheres to the Open-Closed Principle (OCP). Creating interfaces for BurnStatus/PoisonStatus will also demonstrate the Dependency-Inversion Principle (DIP), as those statuses will have a dependency on an interface rather than each of the concrete classes that can be burned/poisoned. This has the added benefit of looser coupling.

While both choices do use downcasting (inevitable with the given engine code), the chosen design uses the engine code to do so, showing integration with the existing system, even if not optimal.

Decision 2b: Poison and burn can be applied to actors (damages every turn). Should they be combined into one interface (e.g. Debuffable)?

The chosen design was to keep them separate. Right now, the effects affect the same actors, so combining would make sense to reduce the number of interfaces and keep it simple. However, if the game was extended to have a fireproof actor, it would need to implement Debuffable so it can be poisoned, but then provide no logic (empty implementation) for being burned. This violates the Interface Segregation Principle (ISP) by forcing it to implement a method (even if empty) it has no concern about. The chosen design avoids this violation by keeping the interfaces separated.

Decision 3: Whether Fire should be implemented as a subclass of Item (EclipseItem) or Ground.

The chosen design was to implement it as an EclipseItem. Obviously, it had to be non-portable (since you can't pick up fire). It burns any actor on it and disappears after 5 turns. All of these requirements can actually be easily solved by the given engine code (e.g. tick method for items on the ground). As items, multiple fires can stack on the same location. On the other hand, implementing it as a ground would require additional implementation to keep track of how many stacks the fire is burning, how many more turns before extinguishing for each stack, and what the ground will return to after the fire extinguished. Implementing Fire as a Ground introduces unnecessary complexity to the logic required to satisfy the requirements and makes it more susceptible to implementation errors. Meanwhile, it is simpler to implement Fire as an EclipseItem, as it integrates well with the engine and simplicity of logic adheres to the KISS principle.

Requirement 3

Decision 1: Making Undead and Slime be able to wander around. This can be done by having the creature choose a random valid spot next to it to move to.

Chosen design: Create WanderBehaviour (implementing the given Behaviour interface) that defines the logic for having any actor choose a random valid spot and returning a MoveActorAction to that location. Undead and Slime then uses this WanderBehaviour.

Advantages	Disadvantages
Avoids code duplication as the wandering for Undead and Slime work exactly the same.	Undead and Slime will be tightly coupled with WanderBehaviour, and cannot function as intended without it.

Alternative design: Let Undead and Slime define the logic for their own behaviour (in their playTurn method).

Advantages	Disadvantages
No additional classes need to be made, so there are less components in this design.	Code needs to be duplicated for Undead and Slime, and each future actor that would have this wandering behaviour.

The chosen design was used as common behaviours can be defined in one place rather than have each individual actor define the same behaviour in their own class. This reduces code duplication, adhering to DRY. This also promotes code reusability since new classes that want to have an existing behaviour can just use the corresponding Behaviour.

Decision 2: Make holes spawn creatures (Undead and Slime).

Chosen design: Have the GameMap (Deprecated99) define the list of creatures holes can spawn.

Advantages	Disadvantages
Allows different GameMaps to customize what creatures their holes can spawn. The same GameMap could also have different holes that spawn different creatures.	Holes cannot be instantiated by a non-parameterized constructor in this design. This means DefaultGroundConstructor cannot be used to add holes. GameMap must replace existing grounds with holes themselves.

Alternative design: Holes have a hardcoded list of creatures they can spawn.

Advantages	Disadvantages
Simpler approach. Holes can be instantiated by a non-parameterized constructor so DefaultGroundCreator can be used to create holes, using what's already given.	Holes cannot change what creature can spawn. If new requirements need a new GameMap with a different list of spawnable creatures, this design must be refactored.

The chosen design was used as it is more easily extensible. Unlike the other design, if a new GameMap needs to be added, with different spawnable creatures, only the new GameMap needs to define what their holes can spawn. The existing Hole class itself requires no modification, therefore adhering to the OCP. Additionally, as this approach allows for dependency injection (Hole class' constructor takes a list of default constructors for Actors*), the Hole class will depend on the abstract Actor class rather than its concrete subclasses (e.g. Slime). This shows looser coupling and adheres to the DIP as well.

*By this I mean List<Supplier<Actor>> will be passed into the constructor for Hole.

Requirement 4

Decision 1: The alarm system.

Trigger: New item that triggers the alarm when picked up.

Consequences:

- All actors hostile to workers (only affects Undead currently) will follow the worker that triggered the alarm until it expires.
- Doors lock themselves and cannot be unlocked until the alarm expires.

Chosen design: Implement the alarm system as a status effect, similar to poison/burn in REQ 2 (with Alarmable interface and AlarmStatus). The picked-up item that triggers the alarm will access every location in the map it's in and apply AlarmStatus effect to each actor/ground/item that reacts to the alarm (i.e. implements the Alarmable interface).

Advantages	Disadvantages
New classes that can react to the alarm just need to implement the interface and define how it reacts to the alarm. No other classes need to be modified.	Iterating through every location in the map to check every actor/ground/item is inefficient.
The alarm status will only be applied across the same map the item is on. The alarm system does not need to be modified when new GameMaps are added to the game.	Like with REQ 2, this still uses downcasting in the engine's code.

Alternative design: Declare attributes for the GameMap used for the game (i.e. Deprecated99), indicating whether it is alarmed and how many turns it lasts. It will also have methods for setting the alarm. Triggering the alarm will have the picked-up item access the GameMap it's on, then downcast to Deprecated99 to set the alarm. Entities that react to the alarm (e.g. doors, undead) will also

access the GameMap every turn and downcast to check if the alarm is active, so they can behave differently.

Advantages	Disadvantages
Simpler to implement, does not introduce new classes.	Since this design downcasts to one specific map (Deprecated99), this does not support new GameMaps being added. To circumvent this, another abstraction layer (e.g. EclipseGameMap) can be placed in between those to handle the alarm. However, the extra abstraction layer increases complexity and violates KISS.
More efficient approach when activating the alarm, as it doesn't need to go through all entities on the map.	-

The chosen design was used as it is more extensible. As mentioned, classes that can react to the alarm can implement the Alarmable interface and define how it reacts to the alarm. No other classes need to be modified.

Furthermore, new GameMaps can be added into the game without needing to modify the alarm system. These adhere to the OCP.

Like with poison/burn in REQ 2, downcasting is used, but done so using the existing engine.

While this design for the alarm system does perform less efficiently, this metric is not the focus of this unit and object-oriented design.